

Proyecto Final – FP Desarrollo de Análisis Web Memoria – Incremento 1

Proyecto: BeerTaste

Alumno: André Bento Nunes

Fecha: 27–31 octubre 2025

Contenido

1. Product Backlog.....	3
2. Sprint 1 Backlog.....	4
3. Descripción del trabajo realizado.....	4
4. Documentación Incremento 1	5
4.1 Consola de arranque de Spring Boot.....	5
4.2 Base de datos en MySQL Workbench.	5
4.3 Pruebas de Endpoints creados	7
4.3.1 Postman – Pruebas unitarias de Endpoints (GET)	7
4.3.2 Postman – Pruebas unitarias de Endpoints (POST).....	9
4.3.3 Postman – Pruebas unitarias de Endpoints (PUT)	11
5. Validaciones y Reglas de Negocio	12
5.1 BeerService.java.....	12
5.2 StyleService.java	12
5.3 UserService.java	13
5.4 EvaluationService.java	13
5.5 CountryService.java.....	13
5.6 Reglas generales	14
6. Estructura del Proyecto.....	14
5. Retrospectiva del Sprint 1	18
6. Plan del siguiente Sprint (Sprint 2)	18

Introducción

El proyecto **BeerTaste** es una aplicación web destinada a registrar y evaluar cervezas de diferentes estilos y países. Permite a los usuarios:

- Consultar cervezas y estilos disponibles.
- Registrar sus evaluaciones de cervezas, incluyendo precio y valoración.
- Gestionar usuarios y asignarles roles (USER o ADMIN).

El proyecto se ha desarrollado usando **Java Spring Boot**, siguiendo una arquitectura **MVC** con separación en capas:

- **Entity**: definición de tablas y relaciones.
- **Repository**: acceso y manejo de datos.
- **Service**: validaciones y reglas de negocio.
- **Controller**: endpoints REST para la interacción con la API.

La primera entrega corresponde al **Incremento 1**, donde se ha implementado la base de datos, CRUD de cervezas, estilos y usuarios, y las validaciones básicas de los servicios.

1. Product Backlog

Descripción:

BeerTaste es una plataforma para la gestión, valoración y exploración de cervezas, donde los usuarios pueden registrar cervezas, estilos, países de origen y realizar evaluaciones.

Historias de Usuario (HU):

ID	Historia de Usuario	Prioridad	Estado
HU-01	Como usuario quiero ver la lista de cervezas disponibles para poder elegir cuál evaluar.	Alta	Implementada
HU-02	Como usuario quiero poder añadir una nueva cerveza (ADMIN) para mantener actualizada la base de datos.	Media	Implementada
HU-03	Como usuario quiero poder editar información de una cerveza (ADMIN) para corregir datos incorrectos.	Media	Implementada
HU-04	Como usuario quiero poder ver los estilos de cerveza disponibles para conocer sus características.	Alta	Implementada
HU-05	Como usuario quiero registrar evaluaciones de cervezas con precio y valoración para compartir mi opinión.	Alta	Implementada
HU-06	Como usuario quiero ver mi historial de evaluaciones para revisar mis opiniones previas.	Media	Parcial
HU-07	Como ADMIN quiero gestionar usuarios (alta, edición) para controlar el acceso y roles.	Media	Implementada
HU-08	Como usuario quiero filtrar cervezas por estilo o país para encontrar las que más me interesen.	Alta	Parcial
HU-09	Como usuario quiero poder acceder a la aplicación de manera segura mediante autenticación básica.	Alta	Implementada

ID	Historia de Usuario	Prioridad	Estado
HU-10	Como usuario quiero ver estadísticas básicas de las evaluaciones para conocer la popularidad de las cervezas.	Baja	Pendiente

2. Sprint 1 Backlog

Objetivo del Sprint 1:

Desarrollar la base del backend del sistema BeerTaste, con persistencia de datos y API REST funcional conectada a una base de datos MySQL.

Requisitos implementados en el Sprint 1:

- Creación de las entidades principales (Beer, Style, Country, User, Evaluation).
- Configuración de base de datos MySQL y conexión mediante JPA.
- Implementación de servicios y repositorios.
- Creación de controladores REST (/api/beers, /api/styles, /api/countries).
- Validaciones básicas en la lógica de negocio.
- Pruebas con Postman de los endpoints.
- Seguridad con usuario en memoria.

3. Descripción del trabajo realizado

Durante este sprint se implementó la estructura completa del backend.

Los principales problemas encontrados fueron:

- Configuración inicial de la conexión con MySQL (resuelto añadiendo dependencias spring-boot-starter-data-jpa y mysql-connector-j).
- Gestión de relaciones entre entidades (@ManyToOne, @OneToMany).
- Validaciones en servicios para evitar inserciones vacías.
- Configuración de seguridad (Spring Security) para permitir el acceso a la API con autenticación básica.

Soluciones:

Se ajustó la configuración en application.properties, se implementaron validaciones en el BeerService, y se probaron todos los endpoints mediante Postman.

4. Documentación Incremento 1

4.1 Consola de arranque de Spring Boot.

```

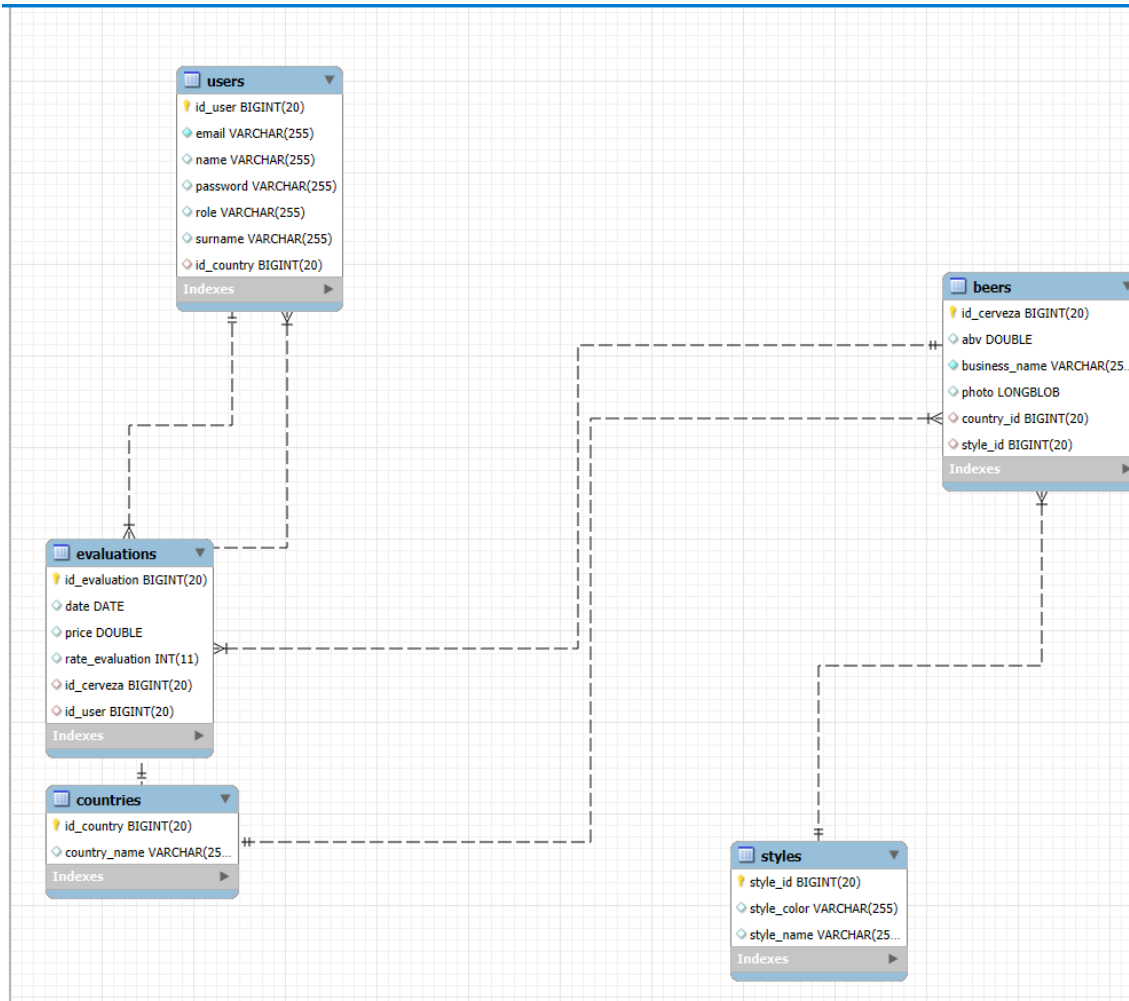
C:\temp\FP\Proyecto\demo > java -jar beertaste.jar
2025-10-29T18:18:04.729+01:00 INFO 35188 --- [dev] [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : For additional web related logging consider setting the 'logging.level.web' property to 'DEBUG'
2025-10-29T18:18:06.206+01:00 INFO 35188 --- [dev] [ restartedMain] s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2025-10-29T18:18:06.319+01:00 INFO 35188 --- [dev] [ restartedMain] s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 98 ms. Found 5 JPA repository interfaces.
2025-10-29T18:18:07.816+01:00 INFO 35188 --- [dev] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2025-10-29T18:18:07.856+01:00 INFO 35188 --- [dev] [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-10-29T18:18:07.856+01:00 INFO 35188 --- [dev] [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.46]
2025-10-29T18:18:08.024+01:00 INFO 35188 --- [dev] [ restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2025-10-29T18:18:08.024+01:00 INFO 35188 --- [dev] [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 3296 ms
2025-10-29T18:18:08.413+01:00 INFO 35188 --- [dev] [ restartedMain] o.hibernate.jpa.internal.util.LogHelper : HHH0000204: Processing PersistenceUnitInfo [name: default]
2025-10-29T18:18:08.536+01:00 INFO 35188 --- [dev] [ restartedMain] org.hibernate.Version : HHH000012: Hibernate ORM core version 6.6.29.Final
2025-10-29T18:18:08.605+01:00 INFO 35188 --- [dev] [ restartedMain] o.h.c.internal.RegionFactoryInitiator : HHH000026: Second-level cache disabled
2025-10-29T18:18:09.183+01:00 INFO 35188 --- [dev] [ restartedMain] o.s.o.j.p.SpringPersistenceUnitInfo : No loadTimeWeaver setup; ignoring JPA class transformer
2025-10-29T18:18:09.254+01:00 INFO 35188 --- [dev] [ restartedMain] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Starting...
2025-10-29T18:18:09.573+01:00 INFO 35188 --- [dev] [ restartedMain] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Start completed.
2025-10-29T18:18:09.573+01:00 INFO 35188 --- [dev] [ restartedMain] org.hibernate.orm.deprecation : HHH0000002: MySQLDialect has been deprecated; use org.hibernate.dialect.MySQLDialect instead
2025-10-29T18:18:09.626+01:00 WARN 35188 --- [dev] [ restartedMain] org.hibernate.orm.deprecation : HHH0000002: MySQLDialect has been deprecated; use org.hibernate.dialect.MySQLDialect instead
2025-10-29T18:18:09.661+01:00 INFO 35188 --- [dev] [ restartedMain] org.hibernate.orm.connections.pooling : HHH0000185: Database Info:
Database JDBC URL [connecting through datasource 'HikariDataSource (HikariPool-1)']
Database driver: undefined/unknown
Database version: 8.0
Automatic mode: undefined/unknown
Isolation level: undefined/unknown
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2025-10-29T18:18:11.649+01:00 INFO 35188 --- [dev] [ restartedMain] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2025-10-29T18:18:11.947+01:00 INFO 35188 --- [dev] [ restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2025-10-29T18:18:12.906+01:00 WARN 35188 --- [dev] [ restartedMain] jpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2025-10-29T18:18:13.498+01:00 WARN 35188 --- [dev] [ restartedMain] org.springframework.boot.autoconfigure.mustache.TemplateResolverConfiguration : Cannot find template location: classpath:/templates/ (please add some templates, check your Thymeleaf configuration, or set spring.thymeleaf.check-template-location=false)
  
```

4.2 Base de datos en MySQL Workbench.

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows a database named 'beertaste_db' with tables 'beers', 'countries', 'evaluations', 'styles', and 'users'. The main editor shows a SQL query that unions data from these tables to create a 'registros' table. The 'Result Grid' at the bottom shows the output of the query, which includes columns 'table' and 'registros' with values for each table.

table	registros
beers	185
countries	10
evaluations	0
styles	9
users	1000

➤ Diagrama de la base de datos



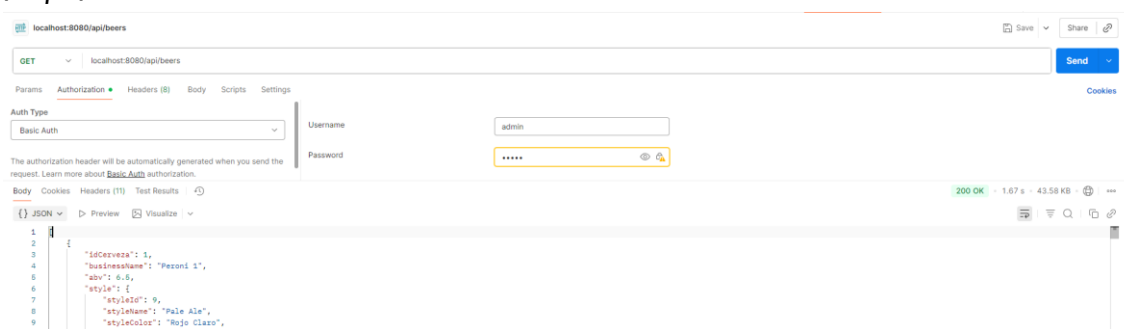
4.3 Pruebas de Endpoints creados

Endpoint	Método	Descripción	Resultado
/api/beers	GET	Lista todas las cervezas	Realizado y Probado
/api/beers/{id}	GET	Muestra detalles de una cerveza	Realizado y Probado
/api/beers	POST	Crea una nueva cerveza	Realizado y Probado
/api/beers/{id}	PUT	Actualiza una cerveza existente	Realizado y Probado
/api/beers/{id}	DELETE	Borra un usuario especifico	Realizado / No Probado
/api/styles	GET	Lista todos los estilos de cerveza	Realizado y Probado
/api/styles/{id}	GET	Lista el estilo epecifico a partir del id	Realizado y Probado
/api/styles	POST	Crea un nuevo estilo	Realizado y Probado
/api/styles/{id}	PUT	Actualiza un estilo de cerca existente	Realizado y Probado
/api/styles/{id}	DELETE	Borra un estilo especifico	Realizado / No Probado
/api/users	GET	Lista todos los usuarios	Realizado y Probado
/api/users/{id}	GET	Lista el usuario especifico a partir de del id	Realizado y Probado
/api/users	POST	Crea un nuevo usuario	Realizado y Probado
/api/users/{id}	PUT	Actualiza el nuevo usuario	Realizado y Probado
/api/users/{id}	DELETE	Borra un usuario especifico	Realizado / No Probado

4.3.1 Postman – Pruebas unitarias de Endpoints (GET)

A continuación, se evidencia las pruebas del correcto funcionamiento para las operaciones de GET para los recursos de beers, styles y users.

/api/beers



/api/beers/{id}

localhost:8080/api/beers/8

GET localhost:8080/api/beers/8

Auth Type: Basic Auth

Username: admin

Password: *****

200 OK · 118 ms · 47.22 KB

```

1 {
2   "idCerveza": 8,
3   "businessName": "Stella Artois 8",
4   "abr": "11.1",
5   "styleId": 1,
6   "styleId": 4,
7   "styleName": "Pilsner",
8   "styleColor": "Amazillo Claro",

```

/api/styles

localhost:8080/api/styles

GET localhost:8080/api/styles

Auth Type: Basic Auth

Username: admin

Password: *****

200 OK · 128 ms · 40.33 KB

```

1 [
2   {
3     "styleId": 1,
4     "styleName": "IPA",
5     "styleColor": "Ambar",

```

/api/styles{id}

localhost:8080/api/styles/6

GET localhost:8080/api/styles/6

Auth Type: Basic Auth

Username: admin

Password: *****

200 OK · 124 ms · 45.6 KB

```

1 {
2   "styleId": 6,
3   "styleName": "Weissbier",
4   "styleColor": "Amazillo Oscuro",

```

/api/users

localhost:8080/api/users

GET localhost:8080/api/users

Auth Type: Basic Auth

Username: admin

Password: *****

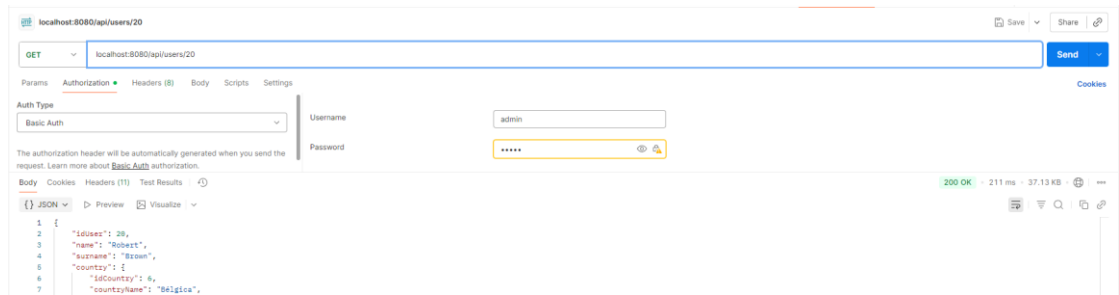
200 OK · 329 ms · 35.45 KB

```

1 [
2   {
3     "idUsers": 1,
4     "name": "Ana",
5     "surname": "Johnson",
6     "country": {
7       "idCountry": 8,
8       "countryName": "Argentina",

```

/api/users/{id}



4.3.2 Postman – Pruebas unitarias de Endpoints (POST)

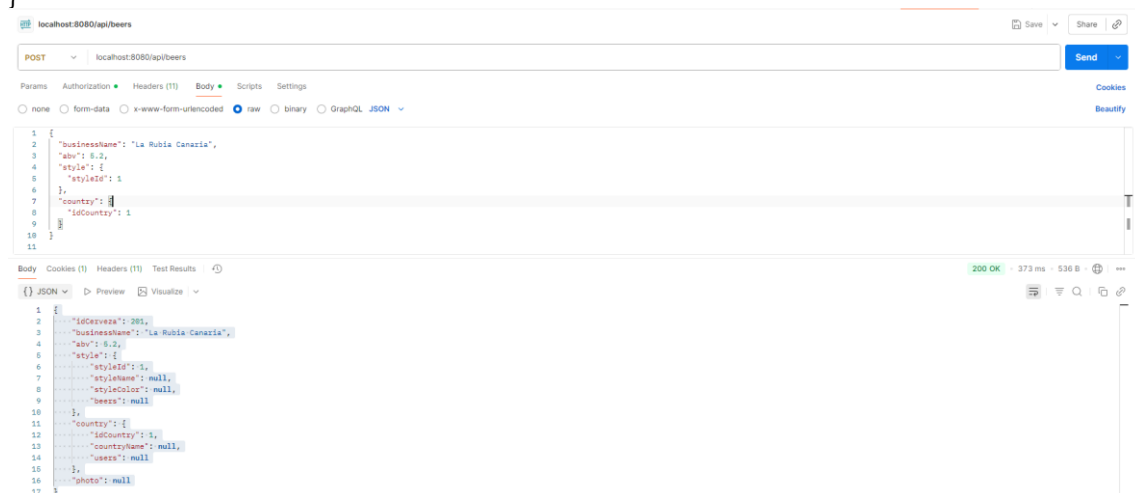
A continuación, se evidencia las pruebas del correcto funcionamiento para las operaciones de POST para los recursos de beers, styles y users.

/api/beers

- Contrato Json

```

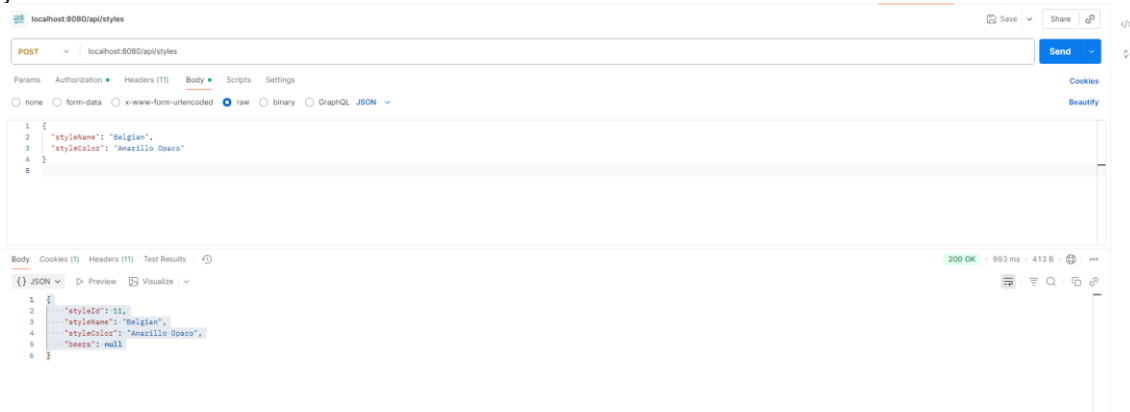
{
  "businessName": "La Rubia Canaria",
  "abv": 5.2,
  "style": {
    "styleId": 1
  },
  "country": {
    "idCountry": 1
  }
}
  
```



/api/sytle

- Contrato Json

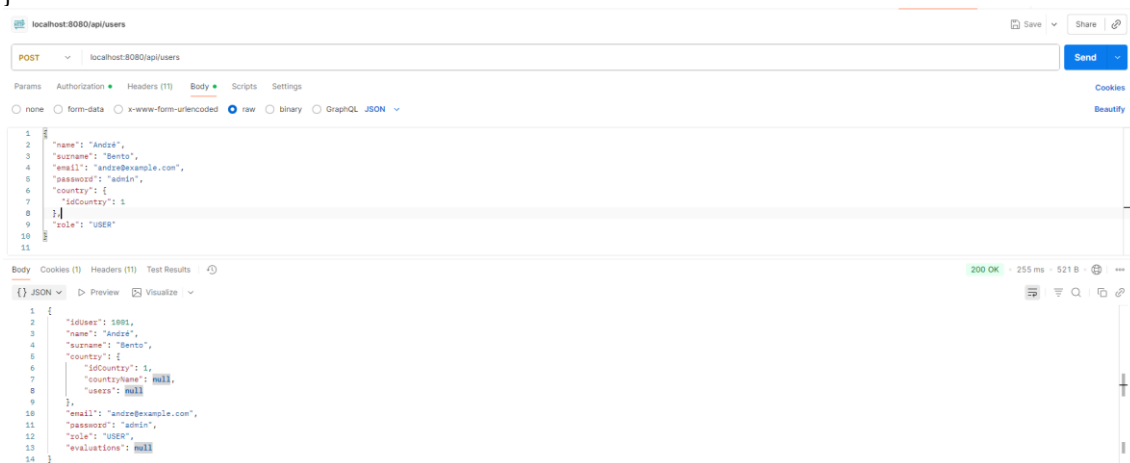
```
{
  "styleName": "Belgian",
  "styleColor": "Amarillo Opaco"
}
```



/api/users

- Contrato Json

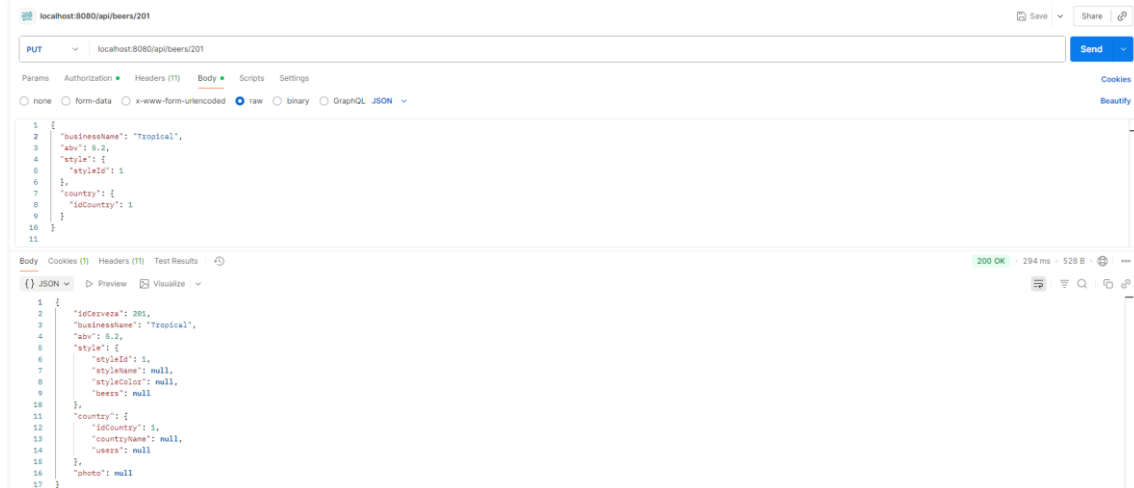
```
{
  "name": "André",
  "surname": "Bento",
  "email": "andre@example.com",
  "password": "admin",
  "country": {
    "idCountry": 1
  },
  "role": "USER"
}
```



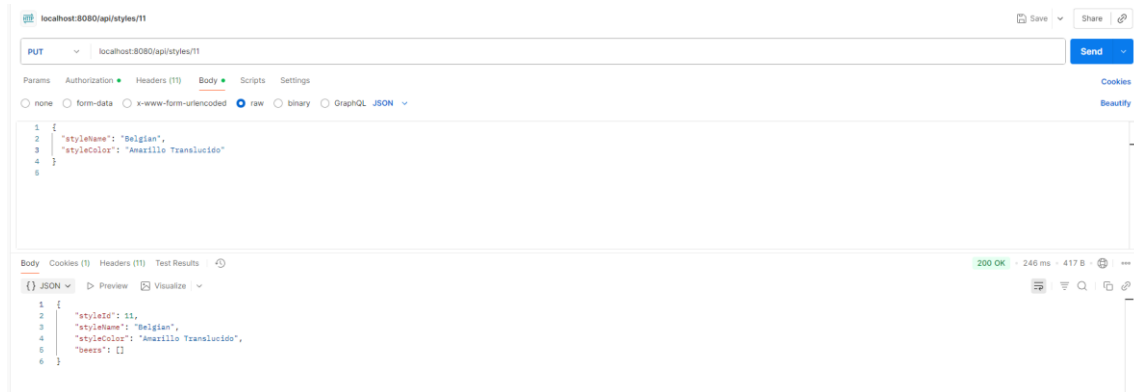
4.3.3 Postman – Pruebas unitarias de Endpoints (PUT)

A continuación, se evidencia las pruebas del correcto funcionamiento para las operaciones de PUT para los recursos de beers, styles y users.

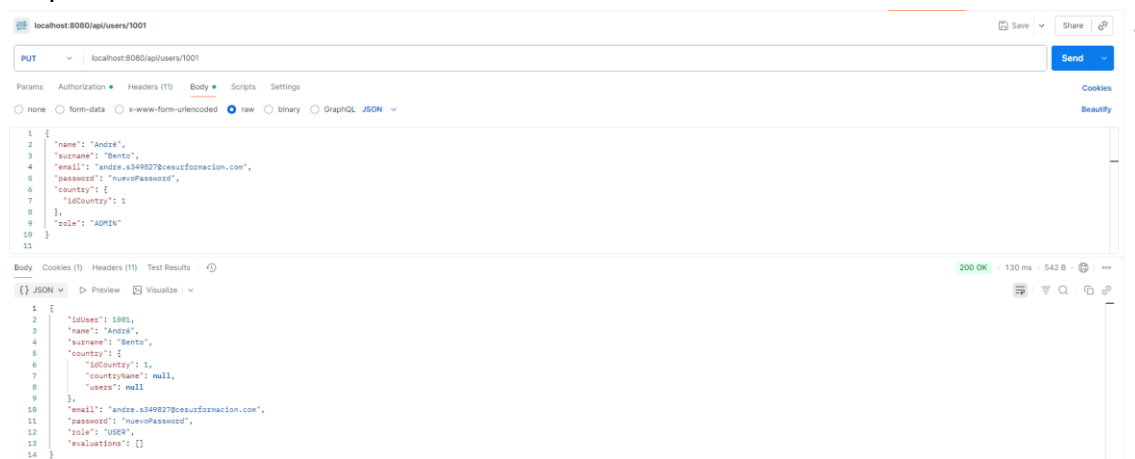
/api/beers



/api/styles



/api/users



5. Validaciones y Reglas de Negocio

5.1 BeerService.java

Validaciones implementadas:

- businessName
 - No puede estar vacío.
 - Se lanza IllegalArgumentException si el valor es null o vacío.
- abv (Alcohol By Volume)
 - Debe ser un número positivo (> 0).
 - Se lanza IllegalArgumentException si es null o negativo.
- style
 - Cada cerveza debe tener un Style asignado.
 - Se lanza IllegalArgumentException si es null.
- Operaciones de eliminación (deleteBeer)
 - Se elimina por id, no requiere validaciones adicionales.

5.2 StyleService.java

Validaciones implementadas:

- styleName
 - No puede estar vacío.
 - Se lanza IllegalArgumentException si es null o vacío.
- styleColor
 - No puede estar vacío.
 - Se lanza IllegalArgumentException si es null o vacío.

Nota: Estas validaciones aseguran que los estilos siempre tengan un nombre y un color definidos.

5.3 UserService.java

Validaciones implementadas:

- name y surname
 - No pueden estar vacíos.
 - Se lanza IllegalArgumentException si alguno es null o vacío.
- email
 - No puede estar vacío y debe ser único en la base de datos.
 - Se lanza IllegalArgumentException si es null, vacío o ya existe otro usuario con el mismo correo.
- password
 - No puede estar vacío.
 - Puede ser almacenada en base64 (según tu diseño), no cifrada en esta versión.
- role
 - Por defecto USER.
 - Solo puede ser USER o ADMIN.

5.4 EvaluationService.java

Validaciones implementadas:

- user
 - No puede ser null.
 - Cada evaluación debe tener un usuario asignado.
- beer
 - No puede ser null.
 - Cada evaluación debe estar vinculada a una cerveza existente.
- price
 - Debe ser un número positivo.
 - Se lanza IllegalArgumentException si es null o negativo.
- rateEvaluation
 - Debe ser un valor entre 1 y 5.
 - Se lanza IllegalArgumentException si el valor está fuera de rango.
- date
 - Por defecto se asigna la fecha actual (LocalDate.now()).

5.5 CountryService.java

- No se permiten modificaciones desde la API (solo GET).
- No hay validaciones porque es una tabla maestra gestionada directamente en la base de datos.

5.6 Reglas generales

- Todas las entidades relacionadas (Beer -> Style, Beer -> Country, Evaluation -> Beer, Evaluation -> User) deben existir antes de asignarlas.
- Se lanza excepción si se intenta guardar una relación con un objeto null.
- Los servicios actúan como capa de validación antes de persistir los datos en el repositorio (Repository).

6. Estructura del Proyecto

Proyecto: BeerTaste

Repositorio GitHub: <https://github.com/annleal/beertaste>

```
| .gitattributes
| .gitignore
| estructura.txt
| HELP.md
| mvnw
| mvnw.cmd
| pom.xml
|
+---.mvn
|   \---wrapper
|         maven-wrapper.properties
|
+---src
|   +---main
|   |   +---java
|   |   |   \---com
|   |   |       \---beertaste
|   |   |           \---demo
|   |   |               DemoApplication.java
|   |   |               ServletInitializer.java
|   |   |
|   |   |   +---config
|   |   |       SecurityConfig.java
|   |   |
|   |   |   +---Controller
|   |   |       BeerController.java
|   |   |       CountryController.java
```

```

| | | | EvaluationController.java
| | | | StyleController.java
| | | | UserController.java
| | | |
| | | +---entity
| | | | Beer.java
| | | | Country.java
| | | | Evaluation.java
| | | | Style.java
| | | | User.java
| | | |
| | | +---repository
| | | | BeerRepository.java
| | | | CountryRepository.java
| | | | EvaluationRepository.java
| | | | StyleRepository.java
| | | | UserRepository.java
| | | |
| | | \---Services
| | | | BeerService.java
| | | | EvaluationService.java
| | | | StyleService.java
| | | | UserService.java
| | | |
| | | \---resources
| | | | application.properties
| | | |
| | | +---static
| | | \---templates
| \---test
| | \---java
| | | \---com
| | | | \---beertaste
| | | | | \---demo
| | | | | DemoApplicationTests.java
|
\---target
| | demo-0.0.1-SNAPSHOT.jar
| | demo-0.0.1-SNAPSHOT.jar.original
|
+---classes

```



```
| | application.properties
| |
| | \---com
| |   \---beertaste
| |     \---demo
| |       DemoApplication.class
| |       ServletInitializer.class
| |
| |       +---config
| |         SecurityConfig.class
| |
| |       +---Controller
| |         BeerController.class
| |         CountryController.class
| |         EvaluationController.class
| |         StyleController.class
| |         UserController.class
| |
| |       +---entity
| |         Beer.class
| |         Country.class
| |         Evaluation.class
| |         Style.class
| |         User.class
| |
| |       +---repository
| |         BeerRepository.class
| |         CountryRepository.class
| |         EvaluationRepository.class
| |         StyleRepository.class
| |         UserRepository.class
| |
| |       \---Services
| |         BeerService.class
| |         EvaluationService.class
| |         StyleService.class
| |         UserService.class
| |
+---generated-sources
|   \---annotations
+---generated-test-sources
```

```
| \---test-annotations
+---maven-archiver
|   pom.properties
|
+---maven-status
|   \---maven-compiler-plugin
|       +---compile
|           | \---default-compile
|           |     createdFiles.lst
|           |     inputFiles.lst
|           |
|       \---testCompile
|           \---default-testCompile
|               createdFiles.lst
|               inputFiles.lst
|
+---surefire-reports
|   com.beertaste.demo.DemoApplicationTests.txt
|   TEST-com.beertaste.demo.DemoApplicationTests.xml
|
\---test-classes
    \---com
        \---beertaste
            \---demo
                DemoApplicationTests.class
```

5. Retrospectiva del Sprint 1

Aspectos positivos:

- Logramos en tener un backend funcional y probado en menos de dos semanas.
- La arquitectura del proyecto es escalable para próximos incrementos.
- Se aplicó correctamente la metodología Scrum en la organización de tareas.

Puntos de mejora:

- Dedicar más tiempo a la documentación visual.
- Planificar mejor la conexión entre backend y frontend para el siguiente sprint.
- Preparar el despliegue en entorno
- Revisar o modificar el backend en función de necesidades del frontend

6. Plan del siguiente Sprint (Sprint 2)

Objetivo: Implementar el frontend (interfaz web) que consuma la API REST y permita la interacción del usuario.

Fecha estimada: 17–21 noviembre 2025.

Tareas previstas:

- Hacer carga inicial en las tablas de evaluations.
- Crear la opción GET para Countries
- Crear la interfaz principal (listado y detalle de cervezas).
- Implementar autenticación en frontend.
- Permitir subir imágenes.
- Conectar con el backend existente.